DATA 260 Homework 9

Pragya Apurva | Rental housing listings

GitHub repository https://github.com/ishuapurva1996/data260-6102

Section 0 - Personal configuration

Setting	Value	Setting	Value
SID4	6102	PORT_BASE	8702
PREFIX	s6102	SEED	6102
VERIFY_SEED	266102	DOMAIN_ID	6

Hardware Apple M4 MacBook Air, 24 GiB RAM
Local model Ollama qwen2.5:7b (Q4_K_M)
Embedding model nomic-embed-text:v1.5
Verified hw9 commit e87064f3e1596fbb239d10472923add0b882213c

Repository access was checked for Sbnikitha and supriyaselvanganesan.

Run log | Metrics | Verification

Part 1 - Redis view counter

Each successful rental detail read increments a prefixed Redis key. The method saves the count to MySQL every 10 views and returns the live count.

Code. rental_views.py lines 33-59

```
30          self.redis_client = redis_client
31          self._increment_if_present = redis_client.register_script(INCREMENT_IF_PRESENT)
32
33      def increment_rental_views(self, db: Session, rental_id: int) -> int | None:
34          key = f"s6102:views:rental:{rental_id}"
35          live_count = self._increment_if_present(keys=[key])
36          if live_count:
37              count = int(live_count)
38          else:
39              # A locking read sees the latest checkpoint even under REPEATABLE READ.
40              saved_count = db.scalar(
41                  select(Rental.view_count).where(Rental.id == rental_id).with_for_update()
42              )
43              if saved_count is None:
44                  return None
45              with self.redis_client.pipeline(transaction=True) as transaction:
46                  transaction.set(key, saved_count, nx=True)
47                  transaction.execute_command("INCR", key)
48                  _, count = transaction.execute()
49
50          if count % 10 == 0:
51              result = db.execute(
52                  update(Rental)
53                  .where(Rental.id == rental_id, Rental.view_count < count)
54                  .values(view_count=count)
55              )
56              db.commit()
57              if result.rowcount:
58                  logger.info("Synced views to database | rental_id=%d count=%d", rental_id, count)
59          return count
60
61
```

Output. 25 views, with MySQL checkpoints at 10 and 20

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

=== Testing Counter | S6102-R-117 ===
Access #01: Total = 1
Access #02: Total = 2
Access #03: Total = 3
Access #04: Total = 4
Access #05: Total = 5
Access #06: Total = 6
Access #07: Total = 7
Access #08: Total = 8
Access #09: Total = 9
  MySQL checkpoint after #9: 0
Access #10: Total = 10
  MySQL checkpoint after #10: 10
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
Access #20: Total = 20
  MySQL checkpoint after #20: 20
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25
  MySQL checkpoint after #25: 20
Final S6102-R-117: Redis=25, MySQL=20

Synced views to database | rental_id=117 count=10
Synced views to database | rental_id=117 count=20
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

Part 2 - Semantic response cache

The Q&A endpoint uses Redis cosine vector search at a cutoff of 0.85. A miss runs the HW8 agents through the local adapter and caches the query, vector, final answer, and data version. A hit returns the saved answer and skips the agents. The handler logs each decision and returns is_cached, similarity, and latency.

Code. kafka_questions.py lines 130-156

```
            )
            timings["embed_ms"] = elapsed(started)
            marker, candidate = await asyncio.to_thread(lookup, vector, user_id, timings)
            is_cached = candidate is not None and candidate["similarity"] >= settings.threshold
            if is_cached:
                answer = candidate["answer"]
            else:
                started = perf_counter()
                facts = await asyncio.to_thread(rental_fact_snapshot, session_factory, question)
                timings["facts_ms"] = elapsed(started)
                fact_marker = facts["dataset_id"], facts["data_version"]
                if marker != fact_marker:
                    candidate = None
                    marker = fact_marker
                started = perf_counter()
                pipeline_attempted = True
                try:
                    async with asyncio.timeout(180):
                        result = await send_question(
                            question,
                            correlation_id=request_id,
                            rental_context=facts,
                        )
                finally:
                    timings["kafka_agent_ms"] = elapsed(started)
                final = result["final"]
                if final["status"] != "approved" or final.get("from_stage") != "reviewer":
                    raise ValueError("Question pipeline did not return an approved final answer.")
                answer = final["answer"]
                started = perf_counter()
                try:
                    cache_write = await asyncio.to_thread(
```

Output. Cache hit and miss from 12 exact, paraphrased, and new queries

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

Pragya Apurva | DATA 260 HW9 Part 2 | Recorded live run
Source 4bc060d11adbd2b8a13bbb525b23cc0280b0ab5d | Redis cosine 0.85

12 HTTP requests | 7 hits (58.33%) | 5 misses | 0 errors
Mean hit 27.445 ms | mean miss 9,012.847 ms

How many units are available at S6102-R-9201?
MISS c19efbee-a3e8-4ce2-af36-92faa71f0572 | 4,017.534 ms
  Kafka inbox offset 2 | final offset 2
  Local generation ran Planner, Writer, Reviewer.
  At S6102-R-9201, there are 3 units available.
HIT  12da59ae-4ae2-4471-b0d4-bf11d7e2ab48 | 32.281 ms
  Same answer from Redis | cosine 1.000000
  No Kafka inbox or final record. Zero generative stage calls.

Across the run: 5 broker journeys, 15 model results, 5 indexed answers.
Cases 7-9 reused a house answer for a different listing and fact.
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

Part 3 - Threshold sweep and cost

cache_cases.yaml was committed before the sweep. It contains 10 exact duplicates, 10 paraphrases, 10 similar questions with different answers, and 10 unrelated questions. The labels stayed fixed.

Code. cache_experiments.py lines 343-374

```
341
342
343  def sweep(runtime, doc):
344      assert (runtime / "seed-reviewed.json").exists(), (
345          "Review the ten live seeds before sweeping"
346      )
347      summaries = []
348      for threshold in THRESHOLDS:
349          namespace = f"c-sweep-{round(threshold * 100):03d}-01"
350          snapshot(runtime, namespace, "restore")
351          rows = []
352          with api(runtime, namespace, threshold) as (client, log):
353              for case in doc["cases"]:
354                  row = request(
355                      client,
356                      case["question"],
357                      runtime / f"requests-{threshold:.2f}.jsonl",
358                  )
359                  body = row["response"]
360                  event = event_for(log, body["request_id"])
361                  assert not event["error"]
362                  assert event["version_enforced"]
363                  assert event["pipeline_attempted"] != body["is_cached"]
364                  row.update(
365                      case_id=case["id"],
366                      category=case["category"],
367                      should_hit=case["should_hit"],
368                      expected=case["expected"],
369                      correct=grade(body["answer"], case["expected"]),
370                      is_cached=body["is_cached"],
371                      cache_event=event,
372                      threshold=threshold,
373                  )
374                  rows.append(row)
375                  append(runtime / "scored-requests.jsonl", row)
```

Output. Five thresholds tested on the same 40 cases

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

 Threshold    Hit rate    False-hit rate    Mean HTTP ms    Cost (sec-equiv)
 0.75         95.0%       52.5%             663.40          6310
 0.80         85.0%       42.5%             2078.91         5130
 0.85         75.0%       32.5%             3832.83         3950
 0.90         72.5%       37.5%             4131.69         4555
 0.95         72.5%       37.5%             4254.44         4555

Frozen case commit 3511ae6
{
  "rule": "Choose the higher threshold among equal minimum total costs.",
  "threshold": 0.85
}
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

The model charges 5 seconds-equivalent per miss for generation and 300 per wrong hit for assumed support time. Choose **0.85** because 10 misses and 13 wrong hits cost **3,950 seconds-equivalent**, the lowest total. Hit and false-hit rates both use 40 requests as the denominator.

Part 3 - False hit and invalidation

Output. Wrong answer reused for a different listing

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

## One observed false hit

- Cached query: How many units are available at S6102-R-9102?
- Incoming query: How many units are available at S6102-R-9106?
- Similarity: 0.999333382; cutoff 0.75.
- Returned cached answer: The rental housing draft is clear and accurate.
Here is the final answer: At S6102-R-9102, there are 4 units available.
- Correct answer: S6102-R-9106 has 1 available unit.
- Likely embedding confusion: the question template and requested field are
identical; only the listing code changes. Their vectors are nearly identical,
so cosine similarity does not preserve record identity.
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

Code. rentals.py lines 119-145 advances the version with the listing update

```
118
119  @router.put("/{rental_id}", response_model=RentalResponse, dependencies=[Depends(can_write)])
120  def update_rental(rental_id: RecordId, payload: RentalUpdate, db: Session = Depends(get_db)):
121      from ..rental_facts import advance_rental_version
122
123      row = find_rental(db, rental_id)
124      if "managerId" in payload.model_fields_set:
125          validate_manager(db, payload.managerId)
126      columns = {
127          "listingTitle": "listing_title",
128          "propertyAddress": "property_address",
129          "submitterEmail": "submitter_email",
130          "description": "description",
131          "propertyType": "property_type",
132          "termsAccepted": "terms_accepted",
133          "listingCode": "listing_code",
134          "availableUnits": "available_units",
135          "managerId": "manager_id",
136      }
137      for key, value in payload.model_dump(exclude_unset=True).items():
138          setattr(row, columns[key], value)
139      row.updated_at = utc_now()
140      try:
141          advance_rental_version(db)
142          db.commit()
143      except IntegrityError as error:
144          raise_write_conflict(db, error, "A rental with this listing code already exists.")
145      return rental_json(row)
146
```

Output. The saved count changed from 3 to 1. The enforced miss and hit return 1.

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

Stale-answer demo | S6102-R-9199 | SQL update from 3 units to 1

 Step            Cached    Data v.    Match v.    Returned answer
 Prime           False     12         None        At S6102-R-9199, there are 3 units available.
 Stale hit       True      13         12          At S6102-R-9199, there are 3 units available.
 Enforced miss   False     13         None        At S6102-R-9199, there is 1 unit available.
 Enforced hit    True      13         13          At S6102-R-9199, there is 1 unit available.
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

Filtering by the current dataset version prevents stale hits. A listing update also invalidates cached answers for unrelated listings.

Part 4.1 - Answer repair

The Evaluator checks the Answering Agent against the reference data and returns PASS or REVISE. On REVISE, the Answering Agent uses the feedback for one more attempt. The second evaluation is final.

Code. Answering Agent, hw09_agents.py lines 106-123

```
        return applicable

    def answer(
        self, question: dict, source: dict, lessons: tuple[Lesson, ...], prior: Attempt | None
    ) -> Answer:
        payload = {"input": question, "listing_source": source}
        if lessons:
            payload["applicable_lessons"] = [lesson.correction for lesson in lessons]
        if prior is not None and prior.grade is not None:
            payload["prior_answer"] = prior.answer.raw
            payload["evaluator_feedback"] = prior.grade.feedback
        result = self.model.complete([("system", ANSWER_PROMPT), ("user", json.dumps(payload))])
        raw = result.content if isinstance(result.content, str) else repr(result.content)
        try:
            claims = json.loads(
                raw, object_pairs_hook=_unique_object, parse_constant=_invalid_constant
            )
        except ValueError:
            claims = None
        return Answer(raw, claims if isinstance(claims, dict) else None)


```

Code. Evaluator Agent, hw09_agents.py lines 138-167

```

        claims = answer.claims
        missing = expected.keys() - claims.keys()
        extras = claims.keys() - expected.keys()
        wrong = {
            field
            for field in expected.keys() & claims.keys()
            if type(claims[field]) is not type(expected[field]) or claims[field] != expected[field]
        }
        if not (missing or extras or wrong):
            return Evaluation("PASS", "All requested fields agree with the reference.")

        fields_to_fix = tuple(sorted(missing | wrong))
        explanation = []
        feedback = []
        if fields_to_fix:
            explanation.append(f"Missing or incorrect: {', '.join(fields_to_fix)}.")
            feedback.append(
                "Set these requested fields to the reference values: "
                + json.dumps({field: expected[field] for field in fields_to_fix})
            )
        if extras:
            unexpected = ", ".join(sorted(extras))
            explanation.append(f"Unexpected fields: {unexpected}.")
            feedback.append(f"Remove unrequested fields: {unexpected}.")
        unsupported = tuple(
            json.dumps({field: claims[field]}, sort_keys=True) for field in sorted(extras | wrong)
        )
        return Evaluation(
            "REVISE", " ".join(explanation), " ".join(feedback), fields_to_fix, unsupported
        )

```

Output. Controlled repair and a separate two-attempt stop

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

4.1  Controlled repair  S6102-R-9402
Tell me the monthly rent and the number of units ready for immediate move-in.
Attempt 1 (availability removed)  {"rent_usd": 1900}
Evaluator  REVISE  |  Missing or incorrect: available_units.
Set these requested fields to the reference values: {"available_units": 4}
Attempt 2  {"available_units": 4, "rent_usd": 1900}
Final  PASS after 2 attempts  |  Rent usd: 1900  |  Available units: 4
Stop control  S6102-R-9401  REVISE -> REVISE  |  Stopped after 2 attempts
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

Part 4.2 - Lesson reuse

Repeated availability errors led to a lesson that reads immediate availability from inventory_export.bucket_b. The agent loads applicable lessons before answering.

Code. part4_agents.py lines 254-266

```
253
254     candidate_path = output / "training_memory/lessons.json"
255     candidate_path.parent.mkdir(exist_ok=True)
256     write_json(candidate_path, [asdict(lesson)])
257     loaded_lessons = tuple(Lesson(**item) for item in read_json(candidate_path))
258     reuse_agent = AnsweringAgent(model, loaded_lessons)
259     reuse = run_cases([train[reuse_question_index]], reference, reuse_agent, grader)[0]
260     if (
261         reuse.lesson_ids != (lesson.lesson_id,)
262         or reuse.attempts[0].grade.result != "PASS"
263         or reuse_agent.use_counts[lesson.lesson_id] != 1
264     ):
265         raise RuntimeError(f"Lesson reuse was not demonstrated: {trace(reuse)}")
266     candidate = replace(lesson, use_count=reuse_agent.use_counts[lesson.lesson_id])
267     write_json(
268         output / "development.json",
```

Output. A later question uses the saved lesson and passes on its first attempt

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

4.2  Later question  S6102-R-9403
How many move-in-ready homes are currently available here?
Loaded rental-ready-units-v1 from lessons.json  |  use_count 0 -> 1
First answer  {"available_units": 1}  |  PASS
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

Datasets and lessons

rental_reference.json | training_data.json | validation_data.json | test_data.json | lessons.json

Part 4.3 and 4.4 - Validation and final results

Paired validation kept the lesson because target results improved, correct answers stayed correct, and unrelated answers gained no unsupported claims. Each variant then ran on the same 10 held-out questions. The small synthetic dataset limits generalization.

Code. part4_agents.py lines 289-306

```
288
289     before = run_cases(validation, reference, AnsweringAgent(model), grader)
290     after = run_cases(validation, reference, AnsweringAgent(model, (candidate,)), grader)
291     accepted, pairs = validate_lesson(before, after, candidate, reference)
292     if development_material()[1] != material_hashes:
293         raise RuntimeError("Development datasets or code changed while the lesson was evaluated")
294     approved = [asdict(candidate)] if accepted else []
295     write_json(output / "lessons.json", approved)
296     write_json(
297         output / "validation.json",
298         {
299             "decided_at": now(),
300             "candidate": asdict(candidate),
301             "accepted": accepted,
302             "before": [trace(run) for run in before],
303             "after": [trace(run) for run in after],
304             "pairs": pairs,
305             "regressions": [pair for pair in pairs if not validation_pair_passes(pair)],
306         },
```

Output. Validation and the four independently measured variants

```
Pragya Apurva | DATA 260 Homework 9 | Saved run output

4.3  Validation retained lesson  |  Target first passes 0/3 -> 2/3  |  Regressions 0

4.4  Held-out results  |  10 identical questions per variant  |  Cache bypassed
Variant                   Correct   Wrong   Attempts      Mean ms
Baseline                        5       5       1.00      1,296.2
Evaluator Repair               10       0       1.50      1,625.0
Lesson Memory                  10       0       1.10      1,313.6
Validated Improvement          10       0       1.10      1,242.6
(base) pragyaapurva@Pragyas-MacBook-Air integration %
```

AI use

1. What you used an AI assistant for and what you did yourself.

In completing this assignment, I used AI as an iterative assistant and reasoning partner. I supplied the assignment requirements, personal configuration, existing application, and scope for each part. I used it to organize modular requirements, explore implementation choices, debug edge cases, and test boundary conditions. Every piece of code and analysis was vetted, refactored, and validated manually to ensure accuracy and alignment with assignment specifications.

2. One unsuitable AI-produced output

The keyword scorer AI helped build passed a cached answer to "What does a co-signer do on a rental lease?" The response explained what a lease is and mentioned tenant responsibilities. The co-signer’s role was missing.

3. How the problem was detected

The words "tenant" and "responsibilities" matched the keyword check. The rubric asked for the co-signer’s guarantee of rent or other obligations if the tenant fails to pay. Reading the full answer against that requirement exposed the gap.

4. What changed and why it works now

The audit recorded two corrected scores alongside the original results. At 0.80, recalculating the table raised the false-hit count from 16 to 17 out of 40 and added 300 seconds-equivalent to the estimated cost. The audit ties that change to the saved answer.